GORDON GROUP MEETING

terco

GPU integrals in standard Fortran: the avenue for GAMESS on GPUs

Jorge Luis Gálvez Vallejo
jorge.galvezvallejo@anu.edu.au

NCI · ANU
Australian National University
Canberra, Australia / October 14, 2026



1 Crawl, walk, run

libERI and GAMESS OpenMP crawled, gmshpc walked, so that terco could run

Crawled: libERI, GAMESS offload

GPU ERIs in GAMESS, then OpenMP target and `do concurrent`.

Alkan et al., JCP 161, 082501; del Angel et al., f functions, JCP 164, 214118

One shell quartet at a time starves the GPU.

Walked: gmshpc

gms++ in CUDA C with four streams, from the Exascale Computing Project.

Binned shell pairs keep the GPU busy, but it is a second code base with a CUDA maintenance burden.

Runs: terco

The same science in standard Fortran `do concurrent`.

No CUDA, no C++. The best ideas from the literature, shown working.

2 How terco works

Standard Fortran only

Kernels are pure `do concurrent` (`nvfortran -stdpar=gpu`). OpenACC only for data, `routine seq` and atomics. The same source builds with gfortran on the CPU.

Binned shell pairs (gmshpc)

Quartets are never stored. Shell pairs are binned by type (l_a, l_b) and Schwarz decade; a bin pair is kept iff $s_{ab} + s_{cd} \leq -\log_{10} \tau$. Each thread derives its quartet from its index.

Integrals per class

Head-Gordon–Pople (VRR, contraction, HRR), 81 classes through d; Rys covers f. Direct 8-fold digestion into J and K.

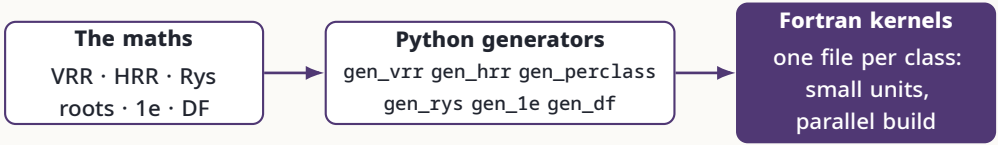
Basis and data

BSE JSON or libcint arrays. Cartesian kernels, spherical on top. Data stays on the device for the whole SCF.

Why bins

At 75 waters/6-31G the stored quartet list cost 26 s of host time and 11.7 GB. Binning by type also sorts by class for free.

2 The kernels are generated, not hand-written



Never edited by hand

A new algorithm is a generator change. CI regenerates the committed kernels and requires them byte for byte.

Rys from the HGP skeleton

Shared decode, screening and digestion; only the integral body differs. Under 1 s to generate, where full HGP text at f took 6 min and 136 MB.

DFT the same way

ExchCXX functionals machine-translated to Fortran: XC on the device, no libxc. **32 run today, wB97X-V included**; the rest say what they need.

3 Performance through d, and f now works

6.8×

faster than libERI

w404, 6-31G (5k basis functions); one Fock build: 22 s vs 149 s

±10%

of gmshpc's CUDA C

w172, 1e-10: 6-31G* 2.26 vs 2.37 s; 6-31G 0.968 vs 0.887 s

0.92×

of GPU4PySCF's time (d)

w20, cc-pVTZ without f (1100 AOs), Fock build: 0.77 vs 0.84 s

Milestone: f functions

- Every class with an f shell is one of **175 generated Rys kernels**; HGP stays at d.
- Matches pyscf on the first build: Fock 8e-13 (water/cc-pVTZ), 2.6e-11 (formamide/def2-TZVP); RHF to 6e-12.
- Roots to 9, so g is reachable. Full GPU build ∼32 min on 4 cores.

cc-pVTZ Fock, 1e-13	terco	GPU4PySCF
w10, f (650 AOs)	0.70 s	0.65 s
w20, f (1300 AOs)	2.20 s	1.44 s
w20, no f (1100)	0.77 s	0.84 s

The f kernels are untuned; tuning has not started.

One V100. Two-electron gradient against GPU4PySCF, 6-31G*: 0.51× its time at w20, 1.24× at w150 (8.41 vs 6.8 s).

4 New: an analytic two-electron gradient

Rys, derivatives by raising angular momentum, D by translational invariance, RHF and UKS-ready, in pure do concurrent. w150 6-31G*, 1e-13:

31.8 s → 8.4 s

1. **Ordered compaction:** screen, then compute only survivors; ket-uniform warps 6% → 68%
2. **Lane-private gradients:** no same-address FP64 atomics; (ss|ss) −47%
3. **Chunked heavy classes:** (pp|pp) 0.64 → 0.13 s, (dp|pp) 1.56 → 0.33 s

Validated, not frozen

pyscf `int2e_ip1` to 1e-12 and finite differences; no test checks terco against its own output. A CUDA twin of the same algorithm was within 1.1–1.3×: the gap left is algorithmic.

Running today, and next

DFT through wB97X-V (range separation, VV10), RKS and UKS, pyscf to ∼3e-11; f energies; MPI and a `bind(c)` ABI.

Next: f tuning, f gradients, meta-GGAs, density fitting.

For GAMESS

GPU HF, DFT and gradients in standard Fortran its developers can read and extend: no CUDA, no C++.

Thank you

GPU HF, DFT and gradients for GAMESS, from a standard-Fortran library its developers can read and extend.

github.com/JorgeG94/terco



Jorge Luis Gálvez Vallejo

jorge.galvezvallejo@anu.edu.au